A3-PS001-TC426

Android Ransomware-like Activity Early Warning in a Controlled Lab

Team CyberSentinel

GUIDE :
Dr. Animesh Giri
Animeshgiri@pes.edu
AIORI Hackathon 3
October 14, 2026

Anupriya
anupriyarao877@gmail.com
AIORI Hackathon 3
October 14, 2026

Anvii S RAI
anviirai22@gmail.com
AIORI Hackathon 3
October 14, 2026

Problem Description

Zero-day Android ransomware cannot be caught by signature-based antivirus, and the Android sandbox stops security tools from inspecting other apps. The one thing it cannot hide is its file-system behavior, so there is a need for a behavior-based early-warning detector that spots it quickly, limits file loss, and stops only the offending workload.

Key Isuues :

- Zero-day ransomware has no known signature, so signature-based antivirus can't detect it
- Android sandboxing doesn't let a security app look into other apps' memory
- Ransomware rewrites and renames a lot of files in just a few seconds, so late detection means more files lost
- The file open, write and rename pattern looks a lot like normal backup, unzip or sync activity, so false positives are a real problem
- Only the controlled test workload should be killed, and the results should be measured using detection delay, precision, recall and files changed

Background

Ransomware has to touch the file system, and Android lets us watch that.

- Ransomware usually lists the files, reads them, overwrites them or writes an encrypted copy, renames them (often with a new extension) and deletes the original [1]
- Android keeps every app in its own sandbox, but documents and media in shared storage can still be reached by apps that have storage permission [2]
- Linux inotify reports file events like open, modify, close-write, create, delete and move [3], and Android's FileObserver is built on it [4], so one designated directory can be watched without touching other apps' memory
- Backup, unzip, photo sync and media compression also write and rename files in bursts, so a simple "too many writes" rule gives false positives
- So we combine several signals (rename rate, extension changes, files rewritten, create-then-delete pairs, entropy) into one score, add hidden canary files for earlier warning, and kill only the allowlisted test workload

Literature & RFC Review

Behaviour-based detection is well studied on desktops, much less on Android.

- CryptoLock [5] stops ransomware by scoring file-type changes, content similarity, entropy and deletions, but it was built for desktop systems
- UNVEIL [6] runs samples in a sandbox and watches file I/O patterns and entropy of written data, close to our telemetry idea but meant for offline analysis, not live alerts
- ShieldFS [7] monitors file-system activity and rolls back encrypted files, but it works at the Windows kernel level
- HelDroid [8] and R-PackDroid [9] are Android-specific, but they classify apps using text, lock-screen and API-package features, not live file bursts
- For standards we use inotify [3], FileObserver [4] and MITRE ATT&CK for Mobile T1471 [10], since no RFC really applies to on-device file monitoring
- Gap: no reproducible Android emulator setup measures detection delay, precision, recall and files changed while killing only the test workload. That is what we build.

Proposed Solution

A behaviour-based detector running in an instrumented Android emulator, with a safe simulator and a scoped responder.

- **Lab setup:** a rooted Android emulator with one designated directory (LabTest) holding generated dummy files and a few hidden canary files
- **Telemetry:** inotify events (open, modify, close-write, create, delete, move) logged with timestamps as JSON lines
- **Detector:** sliding-window features (rename rate, extension changes, files rewritten, create-then-delete pairs, entropy) combined into one score, and a touched canary file raises an alert straight away
- **Responder:** warn at a low threshold, and at a higher one raise the alert and kill only the allowlisted test workload
- **Safe simulation: s**cripted ransomware-like variants (rewrite and rename, rename-only, slow, partial overwrite) confined to LabTest, using a reversible transform, with no network, persistence or ransom note
- **Evaluation:** benign workloads (bulk copy, unzip, atomic save) against simulated attacks, measuring detection delay, precision, recall and files changed

Optimization Proposed

Improvements over a simple "too many writes" detector, before and after.

Aspect	Before (baseline	After (proposed)
Detection trigger	Fixed count of file writes	Weighted score from several features over a sliding window
Early warning	Alert only after many files are already changed	Hidden canary files give an alert on the first touch
False positives	Backup, unzip and media sync look like attacks	Benign workloads used for tuning, plus extension-change and create-then-delete features
Entropy	Used alone, so compressed media gets flagged	One weighted signal among several
Response	Stops processes broadly	Stops only the allowlisted test workload
Threshold tuning	Picked by hand	Threshold sweep on precision and recall

Solution Architecture & Design

Visual : Components of the emulator lab, the detector and the evaluation harness

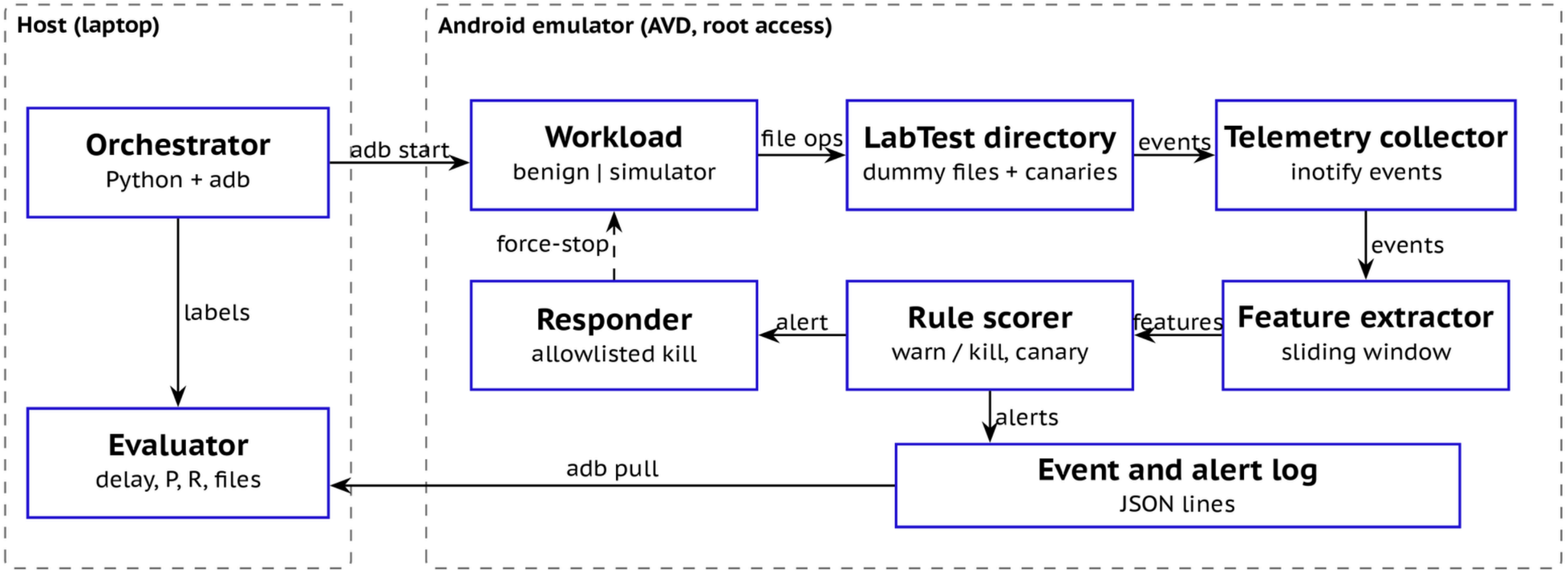


Figure: System Architecture UML Diagram

Data Flow Diagram

Visual:How file events flow to a decision, a response and the results.

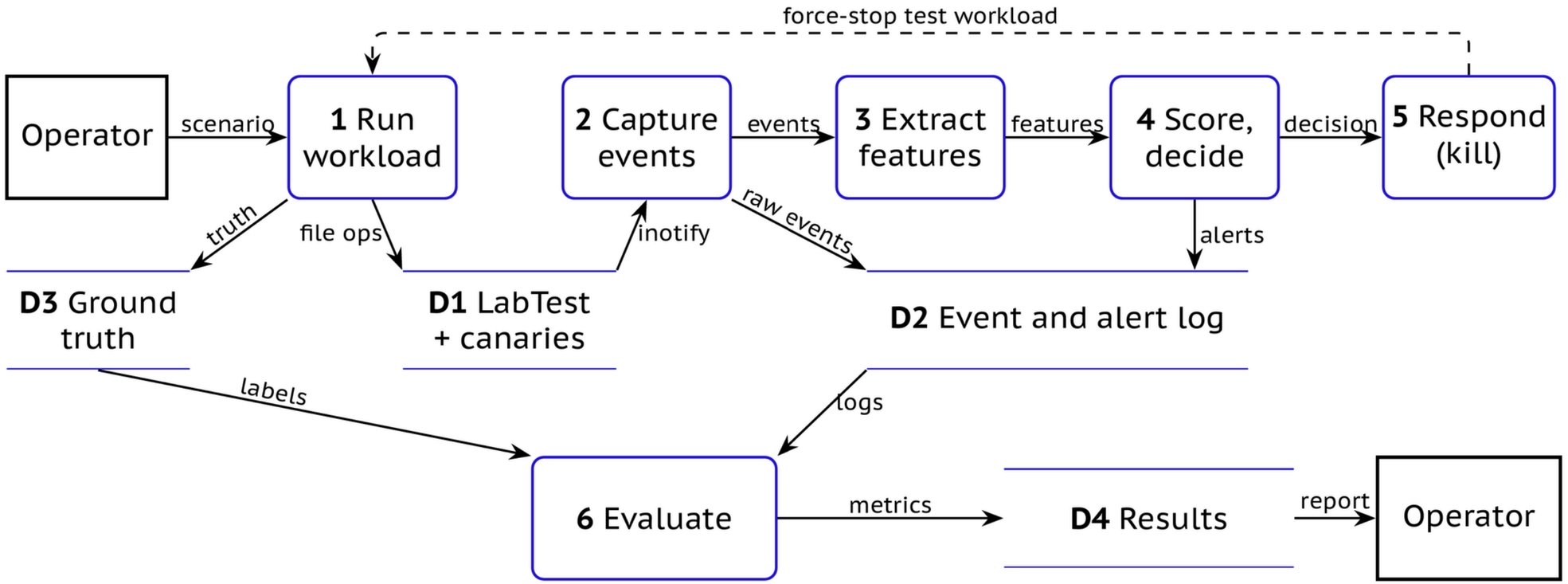


Figure: Data Flow Diagram (DFD)

Figure: Data Flow Diagram (DFD)

Deployment Timeline

Build in small increments, and test each stage in the emulator before starting the next

Step	Phase	Milestone	Output
1	Lab setup	Emulator with root access, LabTest directory, dummy-file generator	Reproducible setup scripts
2	Telemetry	Timestamped inotify capture on LabTest	Event logs
3	Simulation	Safe attack variants, benign workloads, one-command restore	Labelled workloads
4	Detection	Sliding-window score, canary fast path, scoped termination	Detector prototype
5	Evaluation	Repeated runs and threshold sweep	Delay, precision, recall, files changed
6	Demo & documents	Benign and attack demonstration, setup notes	Demo, documentation, slides

Bibliography I

1. Amin Kharraz et al. "Cutting the Gordian Knot: A Look Under the Hood of Ransomware Attacks". In: *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)* (2015).
2. Android Developers. *Security best practices*. https://developer.android.com/privacy-and-security/security-best-practices.
3. Linux man-pages project. *inotify(7)*. https://man7.org/linux/man-pages/man7/inotify.7.html.
4. Android Developers. *FileObserver* (API reference). https://developer.android.com/reference/android/os/FileObserver.
5. Nolen Scaife et al. "CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data". In: *IEEE ICDCS* (2016).
6. Amin Kharraz et al. "UNVEIL: A Large-Scale, Automated Approach to Detecting Ransomware". In: *USENIX Security Symposium* (2016).

Bibliography I

7. Andrea Continella et al. "ShieldFS: A Self-healing, Ransomware-aware Filesystem". In: ACSAC (2016).
8. Nicolò Andronio, Stefano Zanero and Federico Maggi. "HelDroid: Dissecting and Detecting Mobile Ransomware". In: RAID (2015).
9. Davide Maiorca et al. "R-PackDroid: API Package-Based Characterization and Detection of Mobile Ransomware". In: ACM SAC (2017).
10. MITRE. Data Encrypted for Impact (T1471), ATT&CK for Mobile. https://attack.mitre.org/techniques/T1471/.

Thank You for Your Attention!